Descrição dos entregáveis à disciplina de Design Profissional (Profª Isabella Luiza Souza) em parceria com a disciplina de Desenvolvimento Front-End Para Web (Profª Aline Santos).

Entregável I - Escrita de um README.md (com descrição do projeto, público-alvo, funcionalidades, tecnologias utilizadas, requisitos mínimos, instruções para execução e desenvolvedores) do sistema disponibilizado pela professora Aline.

Entregável II - Relatório de Correção de Software (RCS) simplificado do sistema disponibilizado pela professora Aline.

Desenvolvedores

- Nome: AMANDA CARVALHO DE JESUS
- RGM: 48928551
- Nome: GABRIEL MONTEIRO SILVA
- RGM: 7349528513
- Nome: Kaique Andrey
- RGM:

Instruções gerais

1. Utilize este documento como modelo para a elaboração do Relatório de Correção de Software (RCS);
2. Mantenha a formatação original do arquivo, preservando títulos, subtítulos, tabelas e demais elementos estruturais do documento;
3. Substitua todos os textos entre [colchetes] pelas informações do seu grupo;
4. Após preencher cada campo, remova os colchetes, mantendo apenas o conteúdo inserido;
5. A quantidade de bugs documentados não é fixa. As tabelas e seções destinadas ao registro dos problemas poderão ser ampliadas ou reduzidas conforme a quantidade de erros identificados e corrigidos pela equipe;
6. O documento deverá ser preenchido de forma clara, técnica e organizada, registrando as principais etapas de identificação, análise, correção e validação dos problemas encontrados;
7. Após a finalização, o relatório deverá ser exportado e salvo em formato PDF;
8. O arquivo PDF gerado deverá ser incluído e entregue juntamente com os demais artefatos do projeto por meio do repositório no GITHUB;
9. Antes da entrega, revise o documento para verificar se todos os campos obrigatórios foram preenchidos, se não restaram textos de exemplo ou marcações entre colchetes e se as informações apresentadas correspondem às correções efetivamente realizadas no sistema.

Relatório de Correção de Software (RCS)

O Relatório de Correção de Software (RCS) é um documento técnico utilizado para registrar os problemas identificados em um sistema, as análises realizadas, as correções implementadas e os testes executados para validar seufuncionamento. Muito presente nas áreas de Engenharia de Software e Garantia da Qualidade (QA), esse documento também pode ser encontrado com outras denominações, como Relatório de Manutenção Corretiva, Registro de Correção de Defeitos (Bug Fix Report), Documento de Correção de Bugs ou Relatório de Atualização de Software. Seu principal objetivo é garantir a rastreabilidade das alterações realizadas e documentar formalmente o processo de correção do sistema.

1. Identificação do projeto

Sistema analisado

[Mapeamento Vocacional - CareerDreamer]

Objetivo do sistema

[Descobrir na área da tecnologia carreiras que combinem com você.]

Tecnologias utilizadas

[HTML, Css, JavaScript, API, Json, VsCode e Live Server]

2. Levantamento de problemas encontrados

ID	Categoria	Descrição do problema	Severidade
BUG-[01]	Funcionalidade	[O botão não enviava o formulário corretamente]	Alta
BUG-[02]	Funcionalidade	[O campo de nome aceitava apenas números]	Média
BUG-[03]	Interface	[Os campos do formulário estavam organizados de forma incorreta]	Média
BUG-[04]	Responsividade	[O sistema não se encaixava em outros tipos de tela]	Média
BUG-[05]	Interface	[O idioma da página estava configurado incorretamente]	Baixa
BUG-[06]	Funcionalidade	[O formulário não validava corretamente espaços vazios]	Alta

BUG-[07]	Interface	[O contraste entre o texto e o fundo dificultava a leitura]	Baixa
BUG-[08]	Interface	[Os campos não tinham identificação e destaque de foco adequados.	Alta
BUG-[09]	Interface	[O HTML não utilizava elementos semânticos para organizar o conteúdo.]	Média
BUG-[10]	Interface	[Havia erros de escrita nos textos da página]	Baixa
BUG-[11]	Interface	[Os campos precisavam de identificação e indicação visual de foco]	Baixa
BUG-[12]	Funcionalidade	[A mensagem de erro precisava aparecer em uma janela própria]	Média
BUG-[13]	Funcionalidade	[A página precisava organizar as profissões recomendadas por compatibilidade]	Baixa

Descrição das categorias

- Interface: problemas visuais, alinhamento, cores, fontes e organização da página.
- Responsividade: falhas de adaptação em diferentes tamanhos de tela.
- Funcionalidade: botões, menus, formulários ou recursos que não funcionam corretamente.

Descrição dos níveis de severidade

- Baixa: O problema não impede o uso da funcionalidade e possui impacto apenas estético ou de pouca relevância para a experiência do usuário. Exemplos: desalinhamento de elementos, diferenças de cores, fontes inconsistentes ou pequenos problemas visuais.
- Média: O problema afeta a usabilidade ou dificulta a execução de uma tarefa, mas ainda permite que o usuário utilize o sistema por caminhos alternativos. Exemplos: mensagens de erro incorretas, problemas de responsividade em algumas telas ou dificuldades de navegação.
- Alta: O problema impede o funcionamento correto de uma funcionalidade importante do sistema, comprometendo significativamente a experiência do

usuário. Exemplos: botões que não executam ações, formulários que não enviam dados ou menus que não funcionam.

- Crítica: O problema torna uma funcionalidade essencial inutilizável ou impede a utilização de parte significativa do sistema. Exemplos: impossibilidade de acessar páginas importantes, falhas que impedem o cadastro ou o login, ou erros que tornam a aplicação inutilizável em determinados cenários.

3. Análise e correção

BUG-[]01]

- Descrição do problema: [O botão não enviava o formulário corretamente.]
- Causa identificada: [O botão estava configurado com o tipo errado.]
- Correção realizada: [O tipo do botão foi alterado para permitir o envio do formulário.]
- Arquivos alterados:
    - [index.html]

BUG-[02]

- Descrição do problema: [O campo de nome aceitava apenas números.]
- Causa identificada: [O campo estava configurado para receber números, em vez de texto.]
- Correção realizada: [O campo foi alterado para aceitar nomes e recebeu limites de caracteres.]
- Arquivos alterados:
    - [index.html]

BUG-[03]

- Descrição do problema: [Os campos do formulário estavam organizados de forma incorreta.]
- Causa identificada: [O CSS organizava os campos lado a lado, em vez de um abaixo do outro.]
- Correção realizada: [A organização dos campos foi ajustada para facilitar o preenchimento.]
- Arquivos alterados:
    - [css/style.css]

BUG-[04]

- Descrição do problema: [A página não se ajustava corretamente às telas menores.]

- Causa identificada: [A página tinha configurações de tamanho que dificultavam o uso em celulares.]
- Correção realizada: [Foram ajustadas as configurações para melhorar a visualização em telas menores.]
- Arquivos alterados:
    - [index.html]
    - [css/style.css]

BUG-[05]

- Descrição do problema: [O idioma da página estava configurado incorretamente.]
- Causa identificada: [O HTML estava configurado com o idioma inglês.]
- Correção realizada: [O idioma foi alterado para português do Brasil.]
- Arquivos alterados:
    - [index.html]

BUG-[06]

- Descrição do problema: [Havia erros de escrita nos textos da página.]
- Causa identificada: [Algumas palavras estavam com erros de ortografia.]
- Correção realizada: [Os textos incorretos foram ajustados no padrão da língua portuguesa.]
- Arquivos alterados:
    - [index.html]

BUG-[07]

- Descrição do problema: [As cores dificultavam a leitura da página.]
- Causa identificada: [O contraste entre as cores do texto e do fundo não era adequado.]
- Correção realizada: [As cores foram ajustadas para facilitar a leitura.]
- Arquivos alterados:
    - [css/style.css]

BUG-[08]

- Descrição do problema: [Os campos não mostravam bem onde digitar.]
- Causa identificada: [Os campos precisavam de melhor identificação.]
- Correção realizada: [Os nomes dos campos e a indicação de onde digitar foram melhorados.]
- Arquivos alterados:

    - [index.html]
    - [css/style.css]

BUG-[09]

- Descrição do problema: [O código HTML estava desorganizado.]
- Causa identificada: [Algumas partes da página estavam mal organizadas.]
- Correção realizada: [A organização do código foi melhorada.]
- Arquivos alterados:
    - [index.html]

BUG-[10]

- Descrição do problema: [O formulário aceitava campos de hobbies e interesses vazios ou com informações inválidas.]
- Causa identificada: [O sistema não verificava corretamente se os campos estavam preenchidos.]
- Correção realizada: [Foi ajustada a verificação para exigir o preenchimento dos campos e ignorar espaços vazios.]
- Arquivos alterados:
    - [js/app.js]

BUG-[11]

- Descrição do problema: [A mensagem de erro aparecia de forma simples na tela.]
- Causa identificada: [O sistema usava o aviso padrão do navegador.]
- Correção realizada: [Foi criada uma janela de aviso personalizada, com um botão para fechar.]
- Arquivos alterados:
    - [index.html]
    - [css/style.css]
    - [js/app.js]

BUG-[12]

- Descrição do problema: [O formulário não enviava os dados corretamente para a API.]

- Causa identificada: [O envio dos dados e o tratamento dos erros precisavam de ajustes.]
- Correção realizada: [O envio dos dados foi ajustado e foram adicionados avisos durante o carregamento e quando ocorria algum erro.]
- Arquivos alterados:
    - [js/config.js]
    - [js/app.js]

BUG-[13]

- Descrição do problema: [A página de resultados não mostrava as informações corretamente.]
- Causa identificada: [O sistema precisava de ajustes para ler e mostrar os resultados.]
- Correção realizada: [Foi ajustada a exibição do nome, do resumo do perfil e das carreiras recomendadas, organizadas pela compatibilidade.]
- Arquivos alterados:
    - [resultado.html]
    - [js/resultado.js]
    - [css/style.css]

4. Evidências

BUG-[01]

Antes da correção

[O botão estava configurado com o tipo incorreto: <button type="button">Enviar</button>]

Após a correção

[O botão foi alterado para enviar o formulário: <button type="submit">Enviar</button>]

BUG-[02]

Antes da correção

[O campo de nome aceitava apenas números: <input type="number" id="nome">]

Após a correção

[O campo foi alterado para aceitar texto e limitar a quantidade de caracteres: `<input type="text" id="nome" minlength="2" maxlength="60" required>`]

BUG-[03]

Antes da correção

[Os campos estavam organizados lado a lado: .formulario { display: flex; flex-direction: row; }]

Após a correção

[Os campos foram organizados um abaixo do outro: .formulario { display: flex; flex-direction: column; }]

BUG-[04]

Antes da correção

[A página não tinha a configuração correta para telas menores: o código HTML não possuía a configuração de viewport.]

Após a correção

[Foi adicionada a configuração para melhorar a visualização em celulares: <meta name="viewport" content="width=device-width, initial-scale=1.0">]

BUG-[05]

Antes da correção

[A página estava configurada com o idioma inglês: <html lang="en">]

Após a correção

[O idioma foi alterado para português do Brasil: <html lang="pt-BR">]

BUG-[06]

Antes da correção

[Os textos apresentavam erros de escrita: `<h1>Mapeamento Vocasional</h1>`]

Após a correção

[O texto foi corrigido: `<h1>Mapeamento Vocacional</h1>`]

BUG-[07]

Antes da correção

[As cores dificultavam a leitura: `label { color: #cfd3e3; background-color: #ffffff; }`]

Após a correção

[As cores foram ajustadas para melhorar a leitura: `label { color: #333333; background-color: #ffffff; }`]

BUG-[08]

Antes da correção

[O campo não possuía um rótulo associado e o foco estava desativado: `input:focus { outline: none; }`]

Após a correção

[O campo recebeu um rótulo e um destaque ao ser selecionado: <label for="nome">Nome</label> e input:focus { outline: 2px solid blue; }]

BUG-[09]

Antes da correção

[O código utilizava elementos `<div>` para organizar o conteúdo da página: Cabeçalho Conteúdo]

Após a correção

[O conteúdo foi organizado com elementos semânticos: <header>Cabeçalho</header><main>Conteúdo</main><footer>Rodapé</footer>]

BUG-[10]

Antes da correção

[O sistema não removia espaços extras nem ignorava itens vazios nos hobbies e interesses.]

Após a correção

[O sistema passou a remover espaços e ignorar itens vazios: const hobbies = campoHobbies.value.split(",").map(item => item.trim()).filter(item => item !== "");]

BUG-[11]

Antes da correção

[A mensagem de erro utilizava o aviso padrão do navegador: alert("Tente mais tarde");]

Após a correção

[A mensagem passou a utilizar uma janela personalizada: modal.showModal();]

BUG-[12]

Antes da correção

[O sistema apenas mostrava um aviso, sem enviar os dados para a API: alert("Tente mais tarde");]

Após a correção

[O sistema passou a enviar os dados do perfil para a API utilizando o método POST: fetch(CONFIG.API_URL, { method: 'POST', body: JSON.stringify(perfil) });]

BUG-[13]

Antes da correção

[A página não apresentava corretamente os dados do resultado: document.getElementById("resultado").textContent = "";]

Após a correção

[sistema passou a organizar as carreiras recomendadas da maior para a menor compatibilidade, utilizando o código: const carreiras = [...dados.recommended_careers].sort((a, b) => Number(b.match_score) - Number(a.match_score));]

5. Testes realizados

Os testes devem contemplar as correções realizadas.

Teste executado	Resultado esperado	Resultado obtido
[Clicar no botão de envio do formulário após preencher os campos.]	[O formulário deve iniciar o processamento dos dados.]	[O botão funcionou corretamente e iniciou o processamento.]
[Informar um nome com menos de 2 caracteres.]	[O sistema deve impedir o envio e solicitar um nome válido.]	[O sistema identificou o nome inválido e exibiu uma mensagem de aviso.]

[Verificar a disposição dos campos Nome, Hobbies e Áreas de interesse.]	[Os campos devem aparecer organizados verticalmente, facilitando o preenchimento.]	[Os três campos foram apresentados em uma única coluna, conforme a organização esperada.]
[Verificar a página em diferentes tamanhos de tela.]	[O conteúdo deve se adaptar às dimensões da tela, evitando rolagem horizontal.]	[Na tela apresentada, os elementos ficaram organizados. A adaptação em outros tamanhos deve ser conferida.]
[Conferir o idioma definido no documento HTML.]	[O documento deve identificar o português do Brasil como idioma principal.]	[O atributo de idioma foi definido como pt-BR.]
[Verificar os textos e títulos apresentados na página.]	[Os textos devem apresentar a escrita correta, sem erros de digitação.]	[Os textos e termos identificados foram corrigidos e apresentados adequadamente.]
[Observar o contraste entre os textos, os campos e o fundo da página.]	[Os textos devem apresentar contraste suficiente para facilitar a leitura.]	[Os títulos e rótulos aparecem em destaque, mas os textos escuros dentro dos campos precisam de atenção quanto ao contraste.]
[Utilizar a tecla Tab para navegar pelos campos e pelo botão de envio.]	[A navegação deve seguir uma ordem lógica e destacar visualmente o elemento selecionado.]	[O código possui identificação dos campos e tratamento visual de foco para auxiliar a navegação.]
[Conferir a organização estrutural do conteúdo HTML.]	[A página deve utilizar elementos semânticos para organizar cabeçalho, conteúdo principal e demais áreas.]	[A estrutura foi organizada com elementos semânticos, facilitando a compreensão do conteúdo.]
[Tentar enviar o formulário com hobbies ou áreas de interesse vazios.]	[O sistema deve impedir o envio e solicitar o preenchimento dos campos obrigatórios.]	[A validação verifica os campos e impede o envio quando não há informações válidas.]
[Tentar enviar o formulário sem uma API configurada.]	[O sistema deve apresentar uma janela personalizada informando que não foi possível realizar a solicitação.]	[A janela personalizada foi implementada para apresentar a mensagem de aviso ao usuário.]
[Verificar o envio dos dados do formulário para a API.]	[O sistema deve enviar as informações utilizando o método POST e tratar a resposta recebida.]	[O código foi implementado com o método POST, tratamento da resposta e controle de tempo limite.]

[Acessar a página de resultados utilizando os dados de exemplo.]	[O sistema deve apresentar o nome do usuário, a síntese do perfil e as carreiras organizadas por compatibilidade.]	[A página foi preparada para exibir as informações do perfil, as porcentagens de compatibilidade e destacar a carreira com melhor combinação.]

6. Conclusão

Justifique os erros mais difíceis de identificar

[Os erros mais difíceis de encontrar foram os que envolviam o funcionamento do sistema, e não apenas a aparência da página.

O BUG-12 foi um dos mais difíceis porque envolvia o envio das informações para a API e o tratamento das respostas.

O BUG-13 também exigiu bastante atenção, pois foi necessário organizar as carreiras pela porcentagem de compatibilidade e mostrar corretamente o nome do usuário, o resumo do perfil e as carreiras recomendadas.

Já o BUG-08 exigiu cuidado porque foi necessário garantir que o usuário conseguisse navegar pelos campos utilizando o teclado e identificar visualmente qual elemento estava selecionado.

Esses erros foram mais difíceis porque exigiram analisar e ajustar o código para que o sistema funcionasse corretamente.]

Quais conhecimentos de Front-End foram necessários?

[Foram necessários conhecimentos básicos de HTML, CSS e JavaScript.

HTML: para organizar os elementos da página, como títulos, campos do formulário, botões e textos.

CSS: para ajustar o visual da página, as cores, o tamanho dos campos, o posicionamento dos elementos e a adaptação para diferentes telas.

JavaScript: para fazer o formulário funcionar, validar as informações preenchidas, apresentar mensagens de aviso, enviar os dados para a API e exibir os resultados das carreiras.]

O que a equipe aprendeu durante a atividade?

[Durante a atividade, a equipe aprendeu a identificar e corrigir erros no código, melhorar a aparência da página e fazer com que o sistema funcionasse corretamente.

Também aprendemos a importância de testar cada alteração, validar as informações preenchidas pelo usuário e organizar melhor os códigos.]